

Ejercicio de clase 1

Introducción a Python

Prof. Ing. Angel Brito Segura

Fecha de entrega: 22 de agosto de 2026

1. Introducción

Para la realización de programas durante el desarrollo del curso se utilizará el lenguaje de programación **Python** en su versión 3. Este lenguaje es muy popular en la actualidad y tiene una gran cantidad de librerías para realizar cálculos numéricos, análisis de datos, aprendizaje automático, entre otros. Además, es un lenguaje fácil de aprender y utilizar.

Python es un lenguaje interpretado, lo que significa que no es necesario compilar el código antes de ejecutarlo, lo que nos facilita las pruebas y depuración del código línea por línea. Por razones históricas, los lenguajes diseñados para ser utilizados en la interpretación a menudo se denominan *lenguajes de scripting*, mientras que el código fuente de los programas se denominan *scripts*.

No obstante, Python tiene muchas de las características de los lenguajes compilados, por lo que se podría decir que es semi interpretado ya que el código fuente se traduce a un pseudo código máquina intermedio llamado *bytecode* la primera vez que se ejecuta, generando archivos .pyc o .pyo (bytecode optimizado), que son los que se ejecutarán en sucesivas ocasiones.

2. Instalación

Para poder utilizar este lenguaje de programación es necesario contar con un intérprete de Python instalado en la máquina donde estaremos trabajando.

2.1. Local en el Sistema Operativo

Para instalar Python en el sistema operativo local, se puede descargar el instalador desde la [página oficial](#) de Python. Ahí se puede elegir la versión más reciente de Python 3 para el sistema operativo que se esté utilizando (Windows, macOS o Linux).

Es importante asegurarse de que la opción *Add Python to PATH* esté seleccionada durante la instalación para poder ejecutar Python desde la línea de comandos. Una vez instalado, se puede verificar la instalación abriendo una terminal y escribiendo:

```
1 python3 --version
```

Código fuente 1: Verificar instalación de Python

2.2. Google Colab

Google Colab es una plataforma en línea que permite ejecutar código Python en la nube sin necesidad de instalar nada en el sistema local. Para utilizarlo es necesario tener una cuenta de Google y acceder a la página:

<https://colab.research.google.com/>

Una vez dentro de la plataforma, se puede crear un nuevo cuaderno (File -> New notebook in Drive) y comenzar a escribir para ejecutar código Python en las celdas de este notebook. También permite importar archivos desde Google Drive o GitHub para trabajar con datos y proyectos existentes.

2.3. GitHub Codespaces

Para facilitar el desarrollo de código en las actividades del curso, es posible utilizar *GitHub Codespaces*, un entorno de desarrollo en la nube que permite trabajar directamente con los repositorios de GitHub desde un navegador web. De esta manera, no será necesario realizar una instalación local de Visual Studio Code, Python ni de las dependencias requeridas para cada actividad.

Para utilizar GitHub Codespaces, el alumno deberá contar con una cuenta de GitHub. Se recomienda solicitar el acceso al *GitHub Student Developer Pack*, mediante el cual los estudiantes pueden obtener beneficios educativos que incluyen recursos para utilizar GitHub Codespaces.

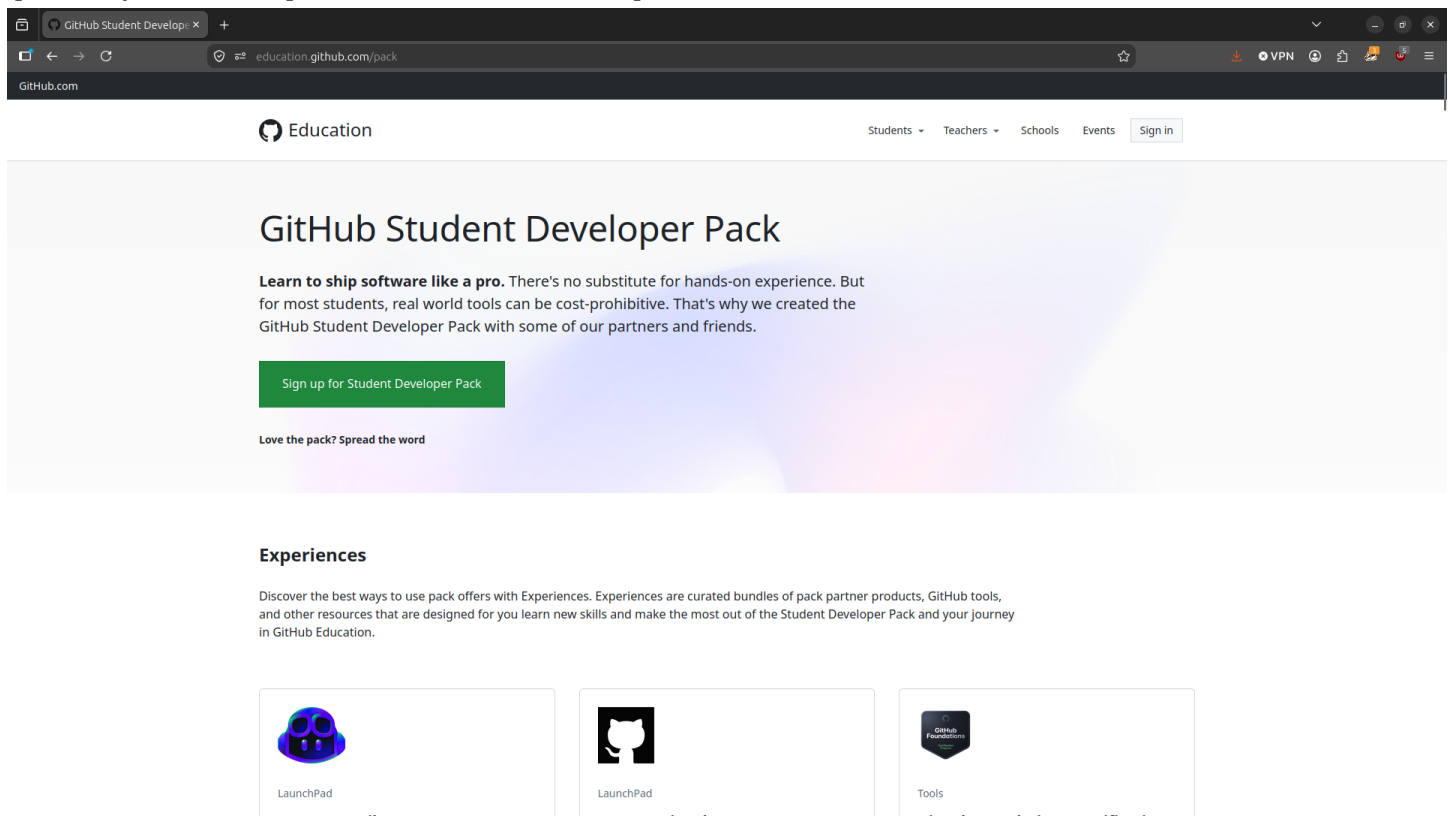


Figura 1: Solicitud a [GitHub Student Developer Pack](#)

Una vez que se ingresa al repositorio correspondiente del entregable a realizar, deberá seleccionar el botón *Code* y posteriormente la opción *Create codespace on main*. GitHub creará un entorno de desarrollo asociado al repositorio, el cual podrá utilizarse directamente desde el navegador.

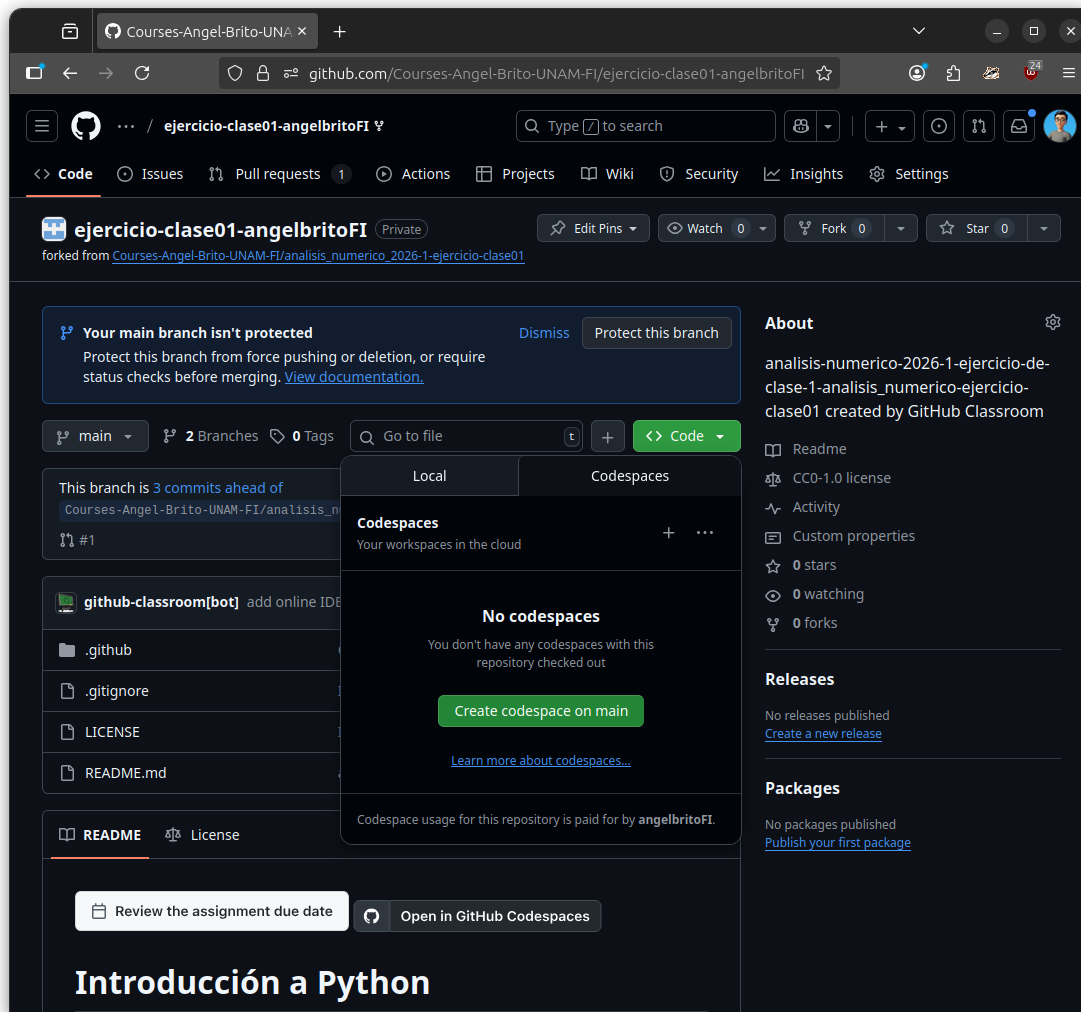


Figura 2: Creación de un entorno de desarrollo mediante GitHub Codespaces

El entorno proporciona una interfaz basada en Visual Studio Code que permite editar los archivos de cada asignación, utilizar la terminal, ejecutar programas y administrar las dependencias necesarias para el desarrollo. Los cambios realizados podrán posteriormente registrarse en el repositorio mediante Git para realizar la entrega correspondiente.

3. Intérprete de Python

El intérprete lee el código fuente de la forma habitual en la cultura occidental: de arriba a abajo y de izquierda a derecha, por lo general. En primer lugar, el intérprete verifica si todas las líneas posteriores son correctas (utilizando los cuatro aspectos cubiertos de cualquier lenguaje) y si el intérprete encuentra un error, finaliza el trabajo inmediatamente y muestra un mensaje de error. Si no hay errores, el intérprete ejecuta las instrucciones en el orden en que aparecen en el código fuente.

4. Primer programa en Python

El primer programa que se suele escribir en cualquier lenguaje de programación es el famoso *Hola Mundo*. En Python, se utiliza la función `print` para mostrar texto en la salida estándar (normalmente la consola o terminal):

```
1 print("Hola Mundo")
```

Código fuente 2: Nuestro primer programa

Una función es una parte separada del código de computadora el cual es capaz de:

- causar algún efecto: enviar texto a la terminal, crear un archivo, dibujar una imagen, reproducir un sonido, etc.
- evaluar un valor y devolverlo como el resultado de la función
- aceptar cualquier cantidad y tipos de argumentos

Por ejemplo, la función `help` nos permite obtener ayuda sobre cualquier función, módulo e incluso sobre las palabras clave existentes en Python:

```
1 help("keywords")
```

Código fuente 3: Obtener las palabras clave en Python

Estas palabras clave mostradas son reservadas y no se deben utilizar como nombres: ni para variables, ni para funciones, ni para cualquier otra cosa que se desee crear. Su significado ya está predefinido y no debe cambiar.

5. Caracteres de escape

Los caracteres de escape son secuencias de caracteres que comienzan con una barra invertida (`\`) y tienen un significado especial en las cadenas de texto. Algunos de los caracteres de escape más comunes en Python son:

- `\n`: Nueva línea
- `\t`: Tabulación
- `\"`: Comillas dobles
- `\'`: Comillas simples
- `\\`: Barra invertida

La palabra `escape` significa que la serie de caracteres en la cadena se escapa por un momento (un momento muy breve) para introducir una inclusión especial. En otras palabras, la barra invertida no significa nada en sí misma, sino que es solo una especie de anuncio de que el siguiente carácter después de la barra invertida también tiene un significado diferente. Por ejemplo, para imprimir un texto en varias líneas, se puede utilizar el carácter de escape `\n`:

```
1 print("<nombre>\n<apellidos>")
```

Código fuente 4: Uso de caracteres de escape

6. Argumentos posicionales y de palabras clave

Los argumentos en las funciones están separados por comas. La forma más común es la posicional, que como su nombre lo indica el significado del argumento está dictado por su posición:

```
1 print("<nombre>", "<apellidos>")
```

Código fuente 5: Argumentos posicionales

Los argumentos de palabras clave (keyword arguments) son aquellos en los que se especifica el nombre del argumento junto con su valor, su nombre proviene del hecho de que el significado de estos argumentos se toma no de su ubicación (posición) sino de la palabra especial (keyword) utilizada para identificarlos, por lo que se pueden pasar en cualquier orden.

Para el caso de la función `print`, se cuenta con el argumento *sep* para indicar la separación que debe tener los argumentos enviado anteriormente (por defecto es un espacio) y el argumento *end* para indicar qué carácter debe imprimirse al final de la línea (por defecto es un salto de línea `\n`):

```
1 print("<nombre>\n<apellido>\t", 5, sep="\n")
2 print("Hola", end="-")
3 print("mUNDO")
```

Código fuente 6: Argumentos de palabras clave

7. Comentarios

Los comentarios son líneas de texto que no son ejecutadas por el intérprete y se utilizan para explicar el código o dejar notas para uno mismo u otros programadores. En Python, los comentarios comienzan con el símbolo `#` y continúan hasta el final de la línea:

```
1 # Esto es un comentario
2 print(5+2) # Esto también es un comentario
```

Código fuente 7: Comentarios en Python

Para comentarios de varias líneas, se pueden utilizar comillas triples (comentario `"` o `'''` comentario `'''`), aunque esta práctica no es muy común en Python:

```
1 """
2 Este es un comentario
3 de varias líneas
4 """
```

Código fuente 8: Comentarios de varias líneas

Para nuestro curso, todos los scripts deben comenzar con el siguiente bloque de comentarios:

```
1 #!/usr/bin/env python3
2 # Autor:           <Nombre completo del alumn>
3 # Fecha de creación: <dd/mm/aaaa>
4 # Descripción:     <Breve descripción de lo que hace el programa>
```

Código fuente 9: Comentarios iniciales para cualquier entregable de código

Para los notebooks, esta información puede estar en la primera celda de texto sin la notación de comentario.

8. Variables y tipos de datos

Los literales son notaciones para representar valores fijos, los cuales se utilizan para codificar datos y ponerlos dentro del código. La forma del literal determina la representación (tipo) que Python utilizará para almacenarlo en la memoria.

```
1 print("2")
2 print(2)
```

Código fuente 10: Representación de números

Para conocer que tipo esta siendo representado en Python, se puede utilizar la función `type`:

```
1 type("2")
```

Código fuente 11: Obtener tipo de dato en Python

Una variable es un nombre que se utiliza para referirse a un valor (literal) almacenado en la memoria. En Python, no es necesario declarar su tipo de dato antes de utilizarla, ya que el intérprete lo infiere automáticamente en función del valor asignado.

Para asignar un valor a una variable, se utiliza el operador de asignación `=`. Por ejemplo:

```
1 texto = "hola"
```

Código fuente 12: Asignación de variable

Estas variables nos sirven para almacenar los resultados de las operaciones que hagamos y con esto poder emplearlas en otras operaciones. Estas cajas (o contenedores) especiales pueden guardar valores en él en casi cualquier forma. Python no impone restricciones en la longitud de los nombres de las variables, pero se recomienda que sea corto y lo más descriptivo posible.

En Python, existen varios tipos de datos básicos, entre los más comunes se encuentran:

```
1 # Números enteros (positivos y negativos)
2 print(+10)
3 print(-1)
4 # Números de punto flotante (decimales)
5 print(2.5)
6 print(-.6)
7 # Cadenas de texto (secuencias de caracteres)
8 print("Darth Vader dijo: \"Yo soy tu padre\"")
9 # Valores booleanos
10 print(True)
11 print(False)
```

Código fuente 13: Tipos de datos en Python

9. Operadores y expresiones

Un operador es un símbolo del lenguaje de programación, el cual es capaz de realizar operaciones con los valores. Cuando los datos y operadores se unen, forman juntos expresiones (la más sencilla es el literal).

```
1 # Suma
2 print(2+2)
3 #Resta
4 print(4. - 8)
5 # Multiplicación
6 print(2 * 3)
7 print(2 * 3.)
8 # Exponenciación
9 print(2 ** 3)
```

```
10 # División (siempre produce un flotante)
11 print(6 / 3)
12 # División entera donde el resultado carece de la parte fraccionaria
13 print(6 // 4)
14 print(6 // 4.)
15 # Obtener el residuo de una división (módulo)
16 print(14 % 4)
17 print(12 % 4.5)
```

Código fuente 14: Operaciones numéricas

Con las cadenas de texto se tienen las siguientes operaciones:

```
1 #Concatenación (enlazado del lado izquierdo, no es conmutativo y ambas deben ser cadenas)
2 var = "3.8.5"
3 pythonV = "Python version: " + var
4 print(pythonV)
5 #Replicación
6 print(5 * "2")
```

Código fuente 15: Operaciones con cadenas de texto en Python

10. Interacción con el usuario

La función `print` solo envía datos a la consola, por lo que no tiene un resultado utilizable. Para poder leer datos que sean introducidos por el usuario y pasar esos datos al programa en ejecución se cuenta con la función `input`:

```
1 nombre = input("Ingresa un nombre")
2 print("Hola", nombre, "!")
```

Código fuente 16: Operaciones con cadenas de texto en Python

Con todo lo visto hasta el momento crea un mensaje de bienvenida personalizado, el cual debe solicitar el nombre del usuario y su edad. El mensaje final debe ser similar a:

Bienvenido <nombre>! Tienes <edad> años cumplidos.

11. Contenido de la entrega

Subir al repositorio todas las sentencias de Python en un archivo fuente (extensión `.py`) o el notebook de Google Colab (extensión `.ipynb`) que se solicitaron en el presente ejercicio de clase.

Seguir la rúbrica general descrita en el documento [syllabus-analisis_numerico_2027-1.pdf](#) y para la entrega del ejercicio de clase seguir el documento [introduccion-git.pdf](#) que se encuentra en el [repositorio](#) de GitHub del curso.